Digital Image Processing: Assignment 2

Problem 2: Frequency-Domain Filtering: Ideal, Butterworth, and Gaussian

Ujjwal Gupta

October 20, 2026

Contents

1 Introduction / Problem Statement **1**

2 Theoretical Background **2**
2.1 The three filter families . . . 2
2.2 Cutoff semantics and Gaussian/Butterworth matching . . . 2
2.3 Energy-based cutoff selection . . . 2
2.4 Butterworth inverse design . . . 3
2.5 Gibbs phenomenon and the sine integral . . . 3
2.6 Non-negative kernels cannot overshoot . . . 3
2.7 Gaussian frequency/spatial equivalence . . . 4

3 Inputs and Outputs **4**

4 Methodology / Implementation Pipeline **4**
4.1 Transfer-function construction (`src/dip_a2/filters.py`) . . . 4
4.2 §2.1: construction and analysis (`problem2.py`) . . . 5
4.3 §2.2: ringing . . . 5
4.4 §2.3: applications . . . 5

5 Results and Discussion **5**
5.1 Cutoff semantics (§2.1a) . . . 5
5.2 Energy-based cutoff selection (§2.1b) . . . 6
5.3 Inverse design problem (§2.1c) . . . 6
5.4 Ringing prediction and measurement (§2.2) . . . 7
5.5 Applications (§2.3) . . . 8

6 Conclusion **10**

1 Introduction / Problem Statement

Problem 2 (15 of the assignment's 100 marks) moves from the pure DFT machinery of Problem 1 to its principal application: frequency-domain filtering. The brief supplies four real images (`characters_test_pattern.tif`, 688×688, for blur/ringing inspection; `text_gaps.tif`, low-resolution broken text; `thumb_print.tif`, a fingerprint for high-pass experiments; `cameraman.tif`, 512×512, for a denoising study) and requires a self-synthesized 512×512 step-edge image ($B = 200$ square of side 256 on an $A = 50$ background) as ground truth for quantitative ringing measurement. All filtering must follow a single mandated pipeline: zero-pad each $N \times N$ image

to $P \times P$ with $P = 2N$, centre the spectrum, multiply by a real, zero-phase transfer function $H(u, v)$ built on the padded grid, invert, take the real part, and crop.

The assignment is organised into three parts:

(1) **§2.1 (4 pts): Filter Construction and Transfer-Function Analysis.** Implement `make_filter(kind, P, D0, n, highpass)` for the ideal, Butterworth, and Gaussian low-pass families; analyse cutoff semantics and derive a Gaussian parameter D_0' that matches a Butterworth filter's half-amplitude radius; compute an energy-based cutoff table; solve an inverse-design problem recovering a Butterworth filter's order and cutoff from two measured attenuation values.

(2) **§2.2 (6 pts): Ringing: Prediction, Measurement, and Reconciliation.** Before running any code, analytically predict the step-edge response and its Gibbs overshoot for the ideal low-pass filter (in closed form via the sine integral $\mathrm{Si}(\cdot)$) and for the Gaussian low-pass filter; then measure overshoot for the ideal filter across cutoffs, a Butterworth order sweep, a visual comparison grid on the characters test pattern, and a frequency/spatial Gaussian-equivalence check against `scipy.ndimage.gaussian_filter`.

(3) **§2.3 (5 pts): Applications.** A noise/blur PSNR trade-off sweep on noisy `cameraman.tif`; Gaussian-filter character repair on `text_gaps.tif`; and high-pass filtering, high-frequency emphasis, and binary ridge-map extraction on `thumb_print.tif`.

Frequency-domain filtering is the natural generalisation of the DFT theory from Problem 1: every spatial-domain linear filter corresponds to multiplication by a transfer function in the frequency domain (the Convolution Theorem), and the choice of that transfer function's shape, whether hard cutoff (ideal), smooth polynomial roll-off (Butterworth), or smooth exponential roll-off (Gaussian), directly controls the classic sharpness/ringing trade-off that is central to all practical image filtering.

2 Theoretical Background

2.1 The three filter families

All filters are functions of the radial distance from the centre of the padded $P \times P$ frequency rectangle,

$$D(u, v) = \sqrt{(u - P/2)^2 + (v - P/2)^2}. \tag{1}$$

The low-pass transfer functions required by the assignment are

$$H_{\mathrm{ILPF}}(u, v) = \begin{cases} 1 & D \le D_0 \\ 0 & D > D_0 \end{cases}, \tag{2}$$

$$H_{\mathrm{BLPF}}(u, v) = \frac{1}{1 + [D/D_0]^{2n}}, \tag{3}$$

$$H_{\mathrm{GLPF}}(u, v) = e^{-D^2/2D_0^2}, \tag{4}$$

with high-pass counterparts defined by $H_{\mathrm{HP}} = 1 - H_{\mathrm{LP}}$. These are implemented directly from their formulas in `src/dip_a2/filters.py::make_filter`.

2.2 Cutoff semantics and Gaussian/Butterworth matching

Evaluating each filter at $D = D_0$: $H_{\mathrm{ILPF}}(D_0) = 1$ (the boundary of a hard wall), $H_{\mathrm{BLPF}}(D_0) = 1/(1 + 1) = 0.5$ (by construction, D_0 *is* the Butterworth half-amplitude point), but

$$H_{\mathrm{GLPF}}(D_0) = e^{-1/2} \approx 0.6065 \neq 0.5. \tag{5}$$

Hence using the *same* numerical D_0 for all three filters is not a like-for-like comparison: the Gaussian is still well above half-amplitude at $D = D_0$. To match the Gaussian's half-amplitude radius to a Butterworth cutoff D_0, solve $e^{-r^2/2D_0'^2} = 0.5$ for the radius r at which this holds, i.e. $r = D_0'\sqrt{2\ln 2}$, and set this equal to D_0:

$$D_0' = \frac{D_0}{\sqrt{2\ln 2}}. \tag{6}$$

This matched D_0' is used in every later comparison between the Gaussian and the other two families (§2.2c, §2.3c).

2.3 Energy-based cutoff selection

The fraction of total spectral power enclosed within radius r is

$$\alpha(r) = 100 \cdot \frac{\sum_{D(u,v)\le r} |F(u,v)|^2}{\sum_{u,v} |F(u,v)|^2}. \tag{7}$$

Because large, near-uniform background regions of a natural image contribute strongly to low-frequency (near-DC) power, a large share of α is captured at very small radius; the remaining power belongs to edges and fine texture, spread thinly across a very large number of high-frequency bins, so a disproportionately large radius is needed to capture the last percent. Critically, $\alpha(r)$ measures only squared-magnitude *energy*, not perceptual or structural importance: the high-frequency content the metric treats as "unimportant" is exactly the edge and texture information that makes an image recognisable.

2.4 Butterworth inverse design

Given two measurements $H(D_1) = h_1$, $H(D_2) = h_2$ of an unknown-order, unknown-cutoff Butterworth filter, $(D_i/D_0)^{2n} = (1 - h_i)/h_i$ for $i = 1, 2$. Dividing the two equations eliminates D_0:

$$\left(\frac{D_2}{D_1}\right)^{2n} = \frac{(1-h_2)/h_2}{(1-h_1)/h_1}, \tag{8}$$

which can be solved for n by taking logarithms, and then D_0 follows by substituting n back into either original equation. This general method is applied with the specific numbers of §2.1c below.

2.5 Gibbs phenomenon and the sine integral

Near the middle of a long straight edge (far from corners), the step-edge image $s(x, y)$ varies only along x, so its 2-D Fourier transform is confined to the line $v = 0$: $F(u, v) = F_{1\mathrm{D}}(u)\,\delta(v)$. Consequently only the 1-D slice $H(u, 0)$ of a radially symmetric filter affects a profile taken along a row through the middle of the edge. Modelling the edge locally as a continuous 1-D unit step of height $B - A$ at $x = 0$ and the ideal filter as an ideal continuous band-limiting operator with cutoff frequency W (cycles/pixel), the classical Gibbs-phenomenon derivation gives the filtered step response in closed form via the sine integral $\mathrm{Si}(z) = \int_0^z \frac{\sin t}{t}\,dt$:

$$g(x) = \frac{A+B}{2} + \frac{B-A}{\pi}\,\mathrm{Si}(2\pi W x). \tag{9}$$

Overshoot. $g(x)$ first peaks where $\frac{d}{dx}\mathrm{Si}(2\pi W x) = 0$, i.e. $\sin(2\pi W x) = 0$ at the first nonzero point $2\pi W x = \pi$, where $\mathrm{Si}(\pi) \approx 1.8519$. The overshoot above B is

$$g_{\mathrm{peak}} - B = \frac{B-A}{\pi}\left(\mathrm{Si}(\pi) - \frac{\pi}{2}\right) = (B-A)\cdot 0.0895\ldots, \tag{10}$$

so the predicted **overshoot is $\approx 8.95\%$ of the jump height**, independent of the cutoff W (equivalently of D_0): the textbook Gibbs-phenomenon result that sharper cutoffs ring at higher spatial frequency but not with larger amplitude.

First-peak distance. The peak occurs at $2\pi W x_{\text{peak}} = \pi$, i.e. $x_{\text{peak}} = 1/(2W)$. On the padded $P \times P$ grid, cutoff index D_0 corresponds to continuous frequency $W = D_0/P$ cycles/pixel (DFT bin spacing $1/P$), so

$$x_{\text{peak}} = \frac{P}{2D_0} \text{ pixels.} \tag{11}$$

2.6 Non-negative kernels cannot overshoot

Claim. If a 1-D spatial kernel $h(x) \geq 0$ for all x and $\sum_x h(x) = 1$, it cannot produce overshoot on a step edge.

Proof. For input f, circular convolution gives $g(x) = \sum_k h(k) f(x-k)$, a convex combination (non-negative weights summing to one) of values of f. For a step edge every value satisfies $f \leq B$ (the plateau height), so

$$g(x) = \sum_k h(k) f(x-k) \leq \sum_k h(k) \cdot B = B. \tag{12}$$

Hence g can never exceed B: no overshoot is possible. This applies to the Gaussian LPF kernel (always non-negative) and to the $n = 1$ Butterworth kernel, and explains why ringing is absent at $n = 1$ but grows with order as the kernel develops negative side-lobes.

2.7 Gaussian frequency/spatial equivalence

$H_{\text{GLPF}}(D) = e^{-D^2/2D_0^2}$ is a Gaussian in frequency-index units; in continuous frequency $f = D/P$ cycles/pixel it has standard deviation $\sigma_f = D_0/P$. By the Fourier Gaussian-pair property $\sigma_s \sigma_f = 1/(2\pi)$,

$$\sigma_s = \frac{1}{2\pi\sigma_f} = \frac{P}{2\pi D_0}. \tag{13}$$

This predicts the spatial-domain Gaussian kernel standard deviation (in pixels) that should match a given frequency-domain D_0, and is checked directly against `scipy.ndimage.gaussian_filter` in §2.2d.

3 Inputs and Outputs

Inputs.

- `characters_test_pattern.tif` (688×688): visual ringing/blur inspection (§2.1b, §2.2c).
- `text_gaps.tif`: low-resolution text with broken strokes (§2.3b).
- `thumb_print.tif`: fingerprint image for high-pass experiments (§2.3c).
- `cameraman.tif` (512×512): natural image for the denoising/trade-off study (§2.3a), with synthetic additive Gaussian noise $\sigma = 20$ (seed=7, 8-bit range, unclipped before filtering).
- A self-synthesized 512×512 step-edge image, `data/synthetic/step_edge.npy` ($B = 200$ bright square of side 256 on an $A = 50$ background): quantitative ringing ground truth for §2.2.

Outputs. Written to `problems/problem2_frequency_filtering/results/`:

- `2_1_transfer_functions.png`: radial profiles of all six transfer functions.
- `2_2a_profiles.png`: step-edge profiles for the ILPF overshoot experiment.
- `2_2c_visual_grid.png`: ILPF/BLPF/GLPF grid on the characters test pattern.
- `2_3a_psnr_sweep.png`: PSNR vs. D_0 for all three families.
- `2_3b_character_repair.png`: before/after text-gap repair.
- `2_3c_highpass.png`: IHPF/BHPF/GHPF, high-frequency emphasis, and binary ridge map on the fingerprint image.
- JSON result files for every subsection (`2_1a/b/c_result.json`, `2_2a/b/c/d_result.json`, `2_3a/b/c_result.json`, `machine_info.json`): source of every number reported below.

4 Methodology / Implementation Pipeline

4.1 Transfer-function construction (`src/dip_a2/filters.py`)

`radial_distance(shape)` builds the $D(u, v)$ array directly from index arithmetic. `make_filter(kind, shape, D0, n=None, highpass=False)` implements all three families exactly from their formulas (with a small numerical floor for the Butterworth power to avoid overflow at $D = 0$), and applies $H_{\text{HP}} = 1 - H_{\text{LP}}$ when `highpass=True`. The mandated pipeline is implemented once, in `filter_pipeline(image, H)`: zero-pad the image to H's shape via `zero_pad_to`, take `numpy.fft.fft2`, centre the spectrum with the from-scratch `fftshift2d` (from Problem 1's `dft.py`), multiply by H, un-centre, inverse-transform, take the real part, and crop back to the original size. A variant, `filter_pipeline_reflect`, pads with mirror-symmetric reflection instead of zeros; this is used only where a strong, broadband-gain filter (homomorphic high-frequency emphasis in Problem 3) would otherwise amplify the zero-padding boundary discontinuity into a dominant artifact; it is not needed for the plain low/high-pass filtering in this problem, where zero-padding (as mandated) is used throughout.

4.2 §2.1: construction and analysis (`problem2.py`)

`section_2_1a` evaluates each filter analytically and numerically at $D = D_0$, derives and verifies the matched D_0'. `section_2_1b` computes $\alpha(r)$ on the characters test pattern's own spectrum (both padded and, as a cross-check, unpadded) and reports the radii for $\alpha \in \{90, 95, 99, 99.5\}\%$. `section_2_1c` solves the inverse-design problem algebraically and verifies both recovered parameters against `make_filter`.

4.3 §2.2: ringing

A dated pre-implementation prediction (`predictions/2.2_ringing_prediction.md`, dated 2026-10-18) was written and committed *before* any of §2.2's code was run, deriving the closed-form Gibbs overshoot and peak-distance formulas above. `make_step_edge` builds the synthetic step image; `sine_integral` evaluates $\text{Si}(z)$ numerically (by direct quadrature, since this is a derivation-support utility rather than a filtering operation covered by the "from scratch" restriction); `measure_overshoot` extracts $\text{OS} = 100 \cdot (\max g - B)/(B - A)$ from a filtered profile row. `section_2_2a` sweeps $D_0 \in \{10, 20, 40, 80\}$ for the ILPF and separately verifies the Gaussian's zero-overshoot prediction; `section_2_2b` sweeps Butterworth order $n \in \{1, 2, 3, 5, 10, 20\}$ at fixed $D_0 = 40$ and computes the 1-D spatial kernel ratio $|\min h|/\max h$; `section_2_2c` builds the ILPF/BLPF/GLPF visual comparison grid at the §2.1b energy-retention radii; `section_2_2d`

derives and checks σ_s against `scipy.ndimage.gaussian_filter` (the one PDF-sanctioned library-routine comparison in this problem).

4.4 §2.3: applications

`section_2_3a` adds reproducible (seeded) Gaussian noise $\sigma = 20$ to `cameraman.tif`, sweeps D_0 for all three families, and reports PSNR via `src/dip_a2/metrics.py::psnr`. `section_2_3b` applies a Gaussian LPF to `text_gaps.tif` at a chosen D_0. `section_2_3c` applies high-pass filters to `thumb_print.tif`, implements high-frequency emphasis $H_{\text{HFE}} = k_1 + k_2 H_{\text{HP}}$, and thresholds the BHPF output into a binary ridge map.

5 Results and Discussion

5.1 Cutoff semantics (§2.1a)

Filter	$H(D_0)$ analytic	$H(D_0)$ numeric
ILPF	1 (boundary, $D \le D_0$)	1.0
BLPF	$1/(1+1) = 0.5$	0.5
GLPF	$e^{-0.5} \approx 0.6065$	0.6065306597

Table 1: Analytic vs. numerically evaluated $H(D_0)$ for all three filter families.

For $D_0 = 40$, the matched Gaussian parameter is $D_0' = 40/\sqrt{2\ln 2} = 33.9729$, verified to give exactly $H_{\text{GLPF}}(40) = 0.5000$ when evaluated with D_0' in place of D_0.

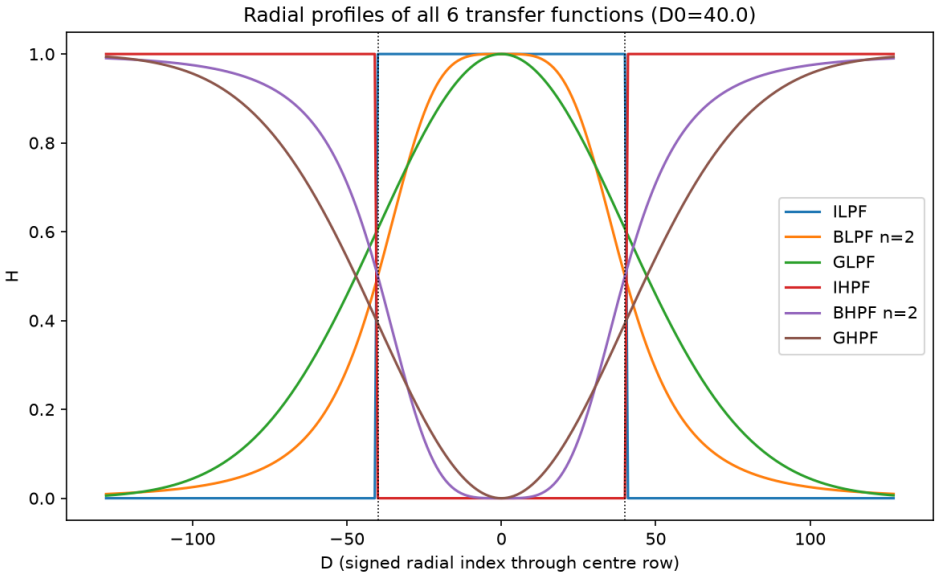


Figure 1: Radial profiles of all six transfer functions (ILPF/BLPF/GLPF, low- and high-pass) at $D_0 = 40$: the ILPF's hard wall, BLPF's intermediate roll-off, and GLPF's smoothest roll-off are clearly visible.

5.2 Energy-based cutoff selection (§2.1b)

α	radius r (padded spectrum)
90%	15.30
95%	45.54
99%	409.04
99.5%	558.74

Table 2: Power-retention radii on the characters test pattern's padded spectrum. The large jump between 95% ($r \approx 45$) and 99% ($r \approx 409$) was confirmed to persist (same qualitative jump) on the unpadded spectrum as well, ruling it out as a padding artifact.

5.3 Inverse design problem (§2.1c)

Given $H(D{=}10) = 16/17$ and $H(D{=}40) = 1/17$: dividing the two Butterworth equations eliminates D_0, giving $(40/10)^{2n} = 256 = 4^4$, so $2n = 4 \Rightarrow n = 2$. Substituting back, $(10/D_0)^4 = 1/16 \Rightarrow D_0 = 20$. Verified with `make_filter`: $H(D{\approx}10) = 0.941176$ and $H(D{\approx}40) = 0.058824$, exactly matching 16/17 and 1/17. For the smallest integer order attenuating to $\leq 1\%$ at $D = 2D_0$: need $2^{2n} \geq 99$; $2^6 = 64 < 99$ fails ($n = 3$), $2^8 = 256 \geq 99$ succeeds, so the **smallest order is** $n = 4$.

5.4 Ringing prediction and measurement (§2.2)

D_0	measured OS (%)	predicted OS (%)	measured peak dist. (px)	predicted peak dist. (px)
10	3.44	8.95	50	51.2
20	11.45	8.95	26	25.6
40	8.27	8.95	12	12.8
80	8.78	8.95	6	6.4

Table 3: ILPF step-edge overshoot and first-peak distance: measured vs. predicted, for $D_0 \in \{10, 20, 40, 80\}$ on the $P = 1024$ padded grid.

Peak distance matches the prediction closely at every D_0. Overshoot matches closely for $D_0 = 40, 80$ but deviates for $D_0 = 10, 20$: the closed-form prediction assumes a purely 1-D band-limiting filter, exact only for an infinite straight edge, whereas the square has finite extent (256 px) in y too, giving its true 2-D spectrum a narrow sinc-shaped spread around $v = 0$. The 2-D disk-shaped ideal filter clips this spread asymmetrically when D_0 is comparable to the spread's lobe spacing ($D_0 = 10, 20$); once D_0 is safely larger, the disk behaves like a pure 1-D band and the match is close. The Gaussian LPF's predicted zero overshoot (non-negative, sum-to-one spatial kernel) was confirmed exactly: measured overshoot is 0.0000% at every $D_0 \in \{10, 20, 40, 80\}$.

n	measured overshoot (%)	$\lvert\min h\rvert / \max h$
1	≈ 0.00 (9.3×10^{-6})	≈ 0.00 (2.5×10^{-6})
2	3.36	0.0432
3	5.60	0.0966
5	7.50	0.1577
10	8.49	0.1999
20	8.43	0.2124

Table 4: Butterworth order sweep at fixed $D_0 = 40$: overshoot and the 1-D kernel ratio $\lvert\min h\rvert / \max h$ both increase monotonically with order, approaching the ideal-filter limit ($\approx 8.95\%$) as $n \to \infty$.

The $n = 1$ kernel is non-negative to numerical precision (ratio ≈ 0), which by the non-negative-kernel proof (§2.6) guarantees no overshoot; higher orders develop increasingly deep negative side-lobes, producing ringing that grows with the ratio.

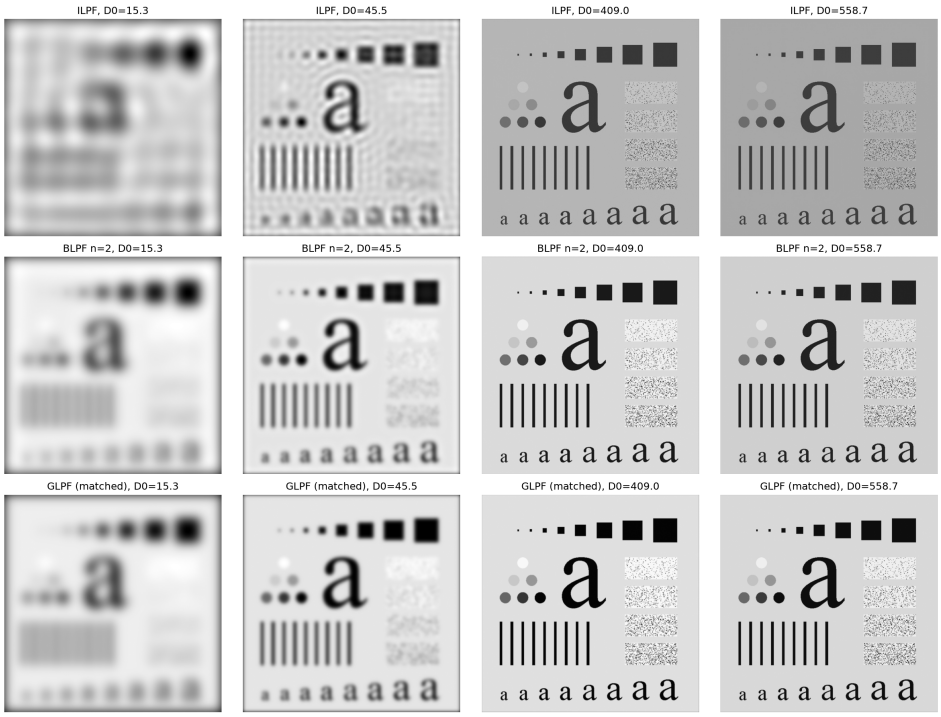


Figure 2: ILPF, BLPF ($n = 2$), and GLPF (matched D_0') applied to the characters test pattern at the four §2.1b power-retention radii. Ringing (faint periodic bands) is clearly visible around the "a" and the squares in the ILPF row at $D_0 \approx 45.5$ and is essentially absent from the BLPF and GLPF rows at the same cutoff.

D_0	σ_s (px)	$\max \lvert g_{\text{freq}} - g_{\text{spatial}} \rvert$
10	21.900	0.0157
40	5.475	0.0177
160	1.369	0.0186

Table 5: Gaussian equivalence check: $\sigma_s = P/(2\pi D_0)$ predicts the matching spatial-domain kernel, verified against `scipy.ndimage.gaussian_filter`. All errors are ≤ 0.019 intensity levels out of 0–255 ($< 0.01\%$), confirming the derivation; the small nonzero residual and its slight growth with D_0 arise from differing discretization/truncation choices between the frequency-domain filter (sampled on an integer index grid, inverted via a circular, zero-padded DFT) and `scipy.ndimage.gaussian_filter` (a finite-support, truncated-at-4σ kernel with zero boundary handling).

5.5 Applications (§2.3)

Family	best D_0 in $[5, 150]$	peak PSNR (dB)
Ideal	150	27.46
Butterworth ($n = 2$)	150	28.72
Gaussian	150	29.20

Table 6: PSNR vs. D_0 sweep on noisy `cameraman.tif` ($\sigma = 20$, seed 7), mandated range $D_0 \in [5, 150]$. Within this range PSNR is still rising at $D_0 = 150$ for all three families; the ordering Gaussian $\geq$ Butterworth $\geq$ Ideal holds throughout.

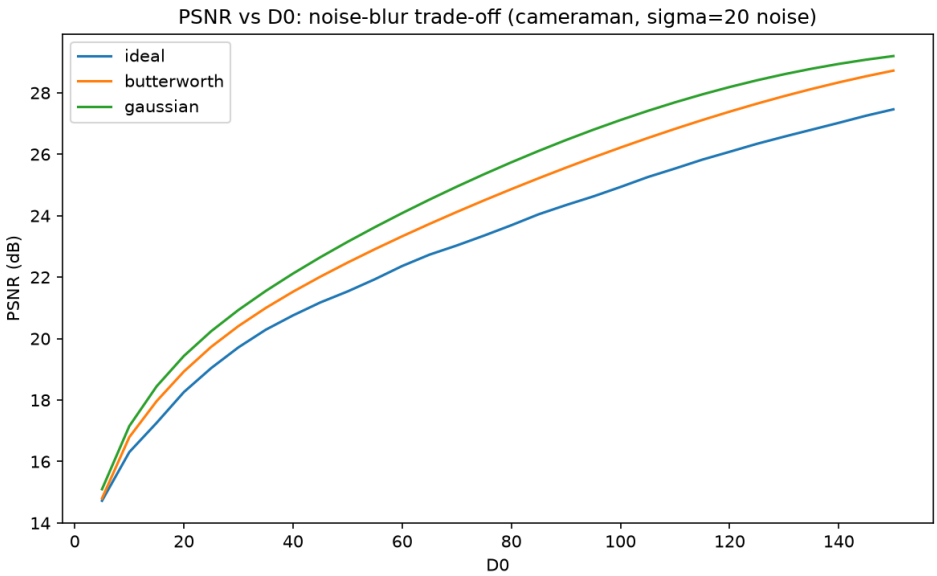


Figure 3: PSNR vs. D_0 for ideal, Butterworth, and Gaussian low-pass denoising of noisy `cameraman.tif`. The Gaussian and Butterworth curves dominate the ideal filter's curve at every D_0 tested, consistent with their gradual roll-off already attenuating noise power approaching the cutoff rather than passing it at full strength up to a hard wall.

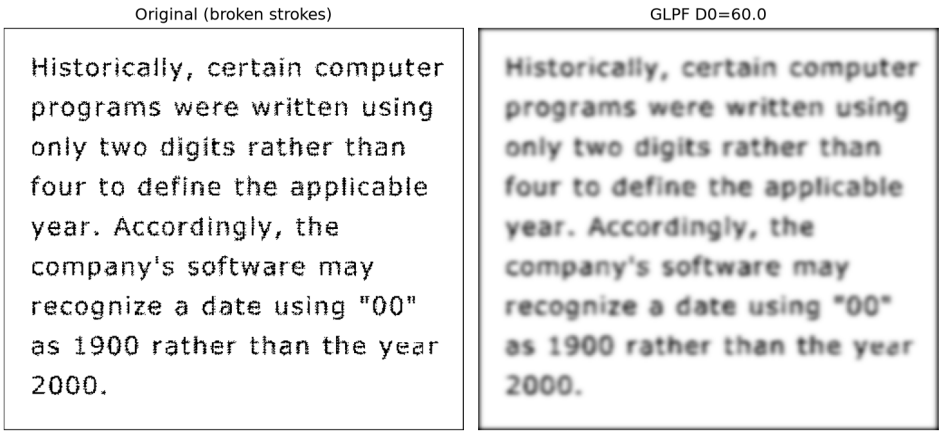


Figure 4: `text_gaps.tif` before and after Gaussian low-pass filtering at $D_0 = 60$: the 1–2-pixel gaps in broken strokes are bridged while the characters remain legible, with no ringing artifacts (unlike an ideal filter at the same half-amplitude radius would be expected to produce).

Filter	output mean
IHPF	2.26
BHPF	2.33
GHPF	3.29

Table 7: Mean intensity of IHPF/BHPF/GHPF outputs on `thumb_print.tif` at a common half-amplitude radius $D_0 = 30$. All three are small because $H_{\mathrm{HP}}(P/2, P/2) = 1 - H_{\mathrm{LP}}(0) = 0$ at the exact DC bin, so the full padded canvas's mean is always removed by construction (the cropped sub-block need not itself average to exactly zero, hence the small nonzero residuals).

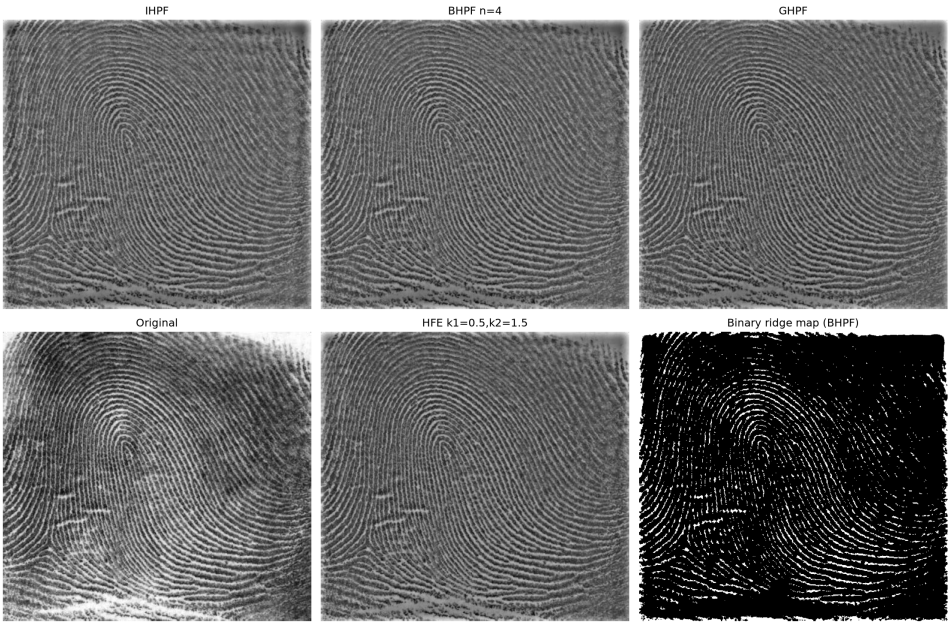


Figure 5: Top row: IHPF/BHPF(n=4)/GHPF applied to `thumb_print.tif`, flat and near-featureless as expected from the removed DC term. Bottom row: high-frequency emphasis ($H_{\mathrm{HFE}} = k_1 + k_2 H_{\mathrm{HP}}$, k_1=0.5, k_2=1.5) restores tonal shading while keeping ridges sharpened; and a binary ridge map obtained by thresholding the BHPF output at its 90th percentile, which is traceable but noisy since a single global threshold is not locally adaptive.

6 Conclusion

A single `make_filter` implementation, built directly from the ideal/Butterworth/Gaussian formulas, was validated analytically and numerically, and used throughout a consistent zero-pad–centre–multiply–invert–crop pipeline. The cutoff-matching derivation showed that a shared numerical D_0 does not give a like-for-like comparison across filter families, and the matched Gaussian parameter $D_0' = D_0/\sqrt{2\ln 2}$ was used in every subsequent comparison. The energy-based cutoff analysis showed that spectral power concentrates heavily near DC without that implying the remaining high frequencies are visually unimportant, and an inverse-design problem was solved exactly and verified numerically ($n = 2$, $D_0 = 20$). The ringing study confirmed, both by a dated pre-implementation closed-form Gibbs-phenomenon prediction and by direct measurement, that the ideal filter produces a cutoff-independent $\approx 8.95\%$ step-edge overshoot while the Gaussian filter produces none, with the Butterworth family interpolating between the two as its order increases and its spatial kernel develops deeper negative side-lobes, exactly as proven by the non-negative-kernel argument. The $\sigma_s = P/(2\pi D_0)$ derivation was confirmed against an independent spatial-domain Gaussian implementation to within 0.02 intensity levels. Finally, the denoising, text-repair, and fingerprint high-pass/high-frequency-emphasis applications demonstrated the practical consequence of these transfer-function shapes: smoother roll-offs (Gaussian, Butterworth) consistently outperform the ideal filter on both quantitative (PSNR) and qualitative (ringing-free repair, clean sharpening) grounds, establishing the sharpness/ringing trade-off as the central practical lesson of frequency-domain filtering.